```
»     let selection = this.selectionTarget.value
»
»     for (let fields of this.additionalFieldsTargets) {
»       fields.disabled = fields.hidden = (fields.dataset.type != selection)
»     }
»   }
  }
```

This has three parts:

- First, we declare a list of targets. Targets identify HTML elements that our controller will interact with. Our targets are a selection element and additional fields. We simply list our targets here without specifying how many of each we expect.

- Next, we define the initialization logic, which could implement as a loop over the targets, hiding each, but it turns out that we can take advantage of the code that shows additional fields. This has the additional benefit of gracefully handing the case where the browser restores the value of some form fields when the user manually refreshes the browser window.

- Finally, we define the code that shows the additional fields. We start by getting the value of the selection. We then iterate over the additional fields. Inside the iteration, we either disable and hide each set of fields or enable and show each set based on whether or not the type of those fields matches the selection.

This all sounds straightforward but won’t completely make sense until we see the markup. So the next step is to define the additional fields.

Defining Additional Fields

Paying online from your checking account involves providing a routing code and an account number. Let’s add these fields to a new partial:

rails80/depot_p/app/views/orders/_check.html.erb